TITLE: INTRODUCTION TO USER-DEFINED FUNCTIONS

OBJECTIVES

- To understand the concept and purpose of functions in programming
- To learn how to declare, define, and call user-defined functions

THEORY

Functions

A **function** is a self-contained block of code designed to perform a specific task.
Functions help make programs modular, easier to read, reusable, and easier to test and debug.

Programs often become large and complex; functions break the program into smaller logical parts.

There are **two types of functions**:

1. **Library Functions**
2. **User-Defined Functions**

Library Functions

Library functions are pre-defined functions provided by programming languages. They are stored in libraries and can be used directly by the programmer without writing their logic.

Examples:

- printf() – prints output (defined in stdio.h)
- scanf() – reads input (defined in stdio.h)
- sqrt() – computes square root (from *math.h*)

User-Defined Functions

User-defined functions are functions that the programmer creates to perform specific tasks required by the program. These functions allow the program to be modular and reusable.

For example, a user may define functions to calculate the area of a circle, to check if a number is prime, to add two numbers etc.

Source Code

```c
#include <stdio.h>

// User-defined function to check even or odd
void CheckEven(int n) {
    if (n % 2 == 0) {
        printf("%d is Even.\n", n);
    } else {
        printf("%d is Odd.\n", n);
    }
}

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    // Function call
    CheckEven(num);

    return 0;
}
```

OUTPUT

```
Enter a number: 22531
22531 is Odd.
```

```
Enter a number: 32420
32420 is Even.
```

TITLE: INTRODUCTION TO RECURSION

OBJECTIVES

- To understand the concept and purpose of recursion in programming
- To learn how to implement recursive functions such as factorial and Fibonacci

THEORY

Recursion

Recursion is a technique in which a function calls itself to solve a problem.
It divides a complex problem into smaller sub-problems of the same type.

Every recursive function must have:

- Base Case
  The condition that stops the function from calling itself repeatedly.
- Recursive Case
  The part where the function calls itself with a smaller input.

When a recursive function is executed, each function call is stored in stack memory. Once the base case is reached, the results are returned step by step until the final result is obtained.

PROBLEMS

Calculate Factorial Using Recursion
The factorial of a number n is defined as:
$n! = n \times (n-1)!$

Base Case:
If $n = 0$ or $n = 1$, factorial is 1. i.e. $n<0$ factorial is 1

Recursive Case:
$\text{factorial}(n) = n \times \text{factorial}(n-1)$

Program to give nth number of Fibonacci Series Using Recursion

The Fibonacci series is defined as:

$F(0) = 0$
$F(1) = 1$
$F(n) = F(n-1) + F(n-2)$

Base Case:
If $n = 0$, return 0
If $n = 1$, return 1

Recursive Case:
$F(n) = F(n-1) + F(n-2)$

Source Code

PROBLEM I : FACTORIAL

```
#include <stdio.h>

int factorial(int n)
{
    if(n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}

int main()
{
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);

    printf("Factorial of %d is %d", num, factorial(num));

    return 0;
}
```

OUTPUT

```
Enter a number: 5
Factorial of 5 is 120
```

PROBLEM II : FIBONACCI NUMBERS

```c
#include <stdio.h>

int fibonacci(int n)
{
    if(n == 0)
        return 0;
    else if(n == 1)
        return 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}

int main()
{
    int n;

    printf("Enter the value of n: ");
    scanf("%d", &n);

    printf("The %dth Fibonacci number is %d", n, fibonacci(n));

    return 0;
}
```

OUTPUT

```
Enter the value of n: 7
The 7th Fibonacci number is 13
```

TITLE : FAMILIARIZATION WITH POINTERS

OBJECTIVES

- To understand the concept of pointers in C
- To learn how to declare, initialize, and use pointers
- To understand the use of pointers in accessing and modifying data

THEORY

Pointers

A pointer is a variable that stores the address of another variable.

Instead of storing a value directly, a pointer stores the memory location where the value is stored.

Pointers are very powerful in C programming. They are used for:

- Dynamic memory allocation
- Passing arguments to functions by reference\
- Arrays and strings handling
- Data structures like linked lists, stacks, and trees

Declaration of Pointer

A pointer is declared using the * symbol.

Syntax:
data_type *pointer_name;
Example:
int *ptr;
This means ptr is a pointer to an integer.

Initialization of Pointer

Before using a pointer, it must be initialized with the address of a variable using the address-of operator (&).

Example:

int num = 10;
int *ptr = #

Here:

- num stores value 10
- &num gives the address of num
- ptr stores the address of num

Accessing Value Using Pointer (Dereferencing)

The * operator is used to access the value stored at the address held by the pointer.

Example:

*ptr

This gives the value stored at the address contained in ptr

Problem I Source code

```c
#include <stdio.h>

int main() {
    int a = 10;          // A normal integer variable
    int *p;              // Pointer declaration

    p = &a;              // Storing address of variable 'a' in pointer 'p'

    printf("Value of a: %d\n", a);
    printf("Address of a: %p\n", &a);

    printf("Value of p (address stored in p): %p\n", p);
    printf("Value pointed to by p (*p): %d\n", *p);

    // Changing value of 'a' using pointer
    *p = 20;
    printf("New value of a after *p = 20: %d\n", a);

    return 0;
}
```

Problem I OBSERVATIONS

Concept	Demonstration
Pointer declaration	`int *p;`
Storing address	`p = &a;`
Accessing value through pointer	`*p`
Modifying variable through pointer	`*p = 20;`
Address printing	`%p` used to show memory addresses

Problem II source code

```c
#include <stdio.h>

// Function to add two numbers using pointers
int add(int *x, int *y) {
    return (*x + *y);
}

// Function to multiply two numbers using pointers
int multiply(int *x, int *y) {
    return (*x * *y);
}

int main() {
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    int sum = add(&a, &b);          // Passing addresses
    int product = multiply(&a, &b); // Passing addresses

    printf("Sum = %d\n", sum);
    printf("Product = %d\n", product);

    return 0;
}
```

Problem II Observations

Task	How it is done
Passing values to function	Addresses of variables (`&a`, `&b`) are passed
Inside function	Parameters are pointers (`int *x`, `int *y`)
Accessing value	Using dereferencing (`*x`, `*y`)

Calculations `(*x + *y)`, `(*x * *y)`

Problem III and IV

Source code

```c
#include <stdio.h>

int main() {
    int c, *pc;

    pc = c;       // error
    *pc = &c;     // error
    pc = &c;      // not an error
    *pc = c;      // not an error

    printf("%d\n", *pc);

    return 0;
}
```

```c
#include <stdio.h>

int main() {
    int c = 5;
    int *p = &c;

    printf("%d\n", *p);

    return 0;
}
```

Question Answers

Why are there errors in problem III?

Code line	Error or no error	Simple reason
`pc = c;`	Error	`pc` is a pointer and can store only an address. `c` is an integer value, not an address.
`*pc = &c;`	Error	`*pc` refers to an integer location, but `&c` is an address. An address cannot be stored in an integer location.
`pc = &c;`	Not an error	`pc` is a pointer, and `&c` is an address. Storing an address in a pointer is correct.
`*pc = c;`	Not an error	`*pc` refers to an integer location, and `c` is an integer value. Storing an integer in an integer location is correct.

Why isn't there an error in problem IV?

In part (b), there is no error because the pointer is used correctly. The variable `c` is an integer, and `p` is declared as a pointer to an integer. When we write `p = &c;`, we store the address of `c` in the pointer `p`, which is the correct use of a pointer. After that, `*p` means the value stored at that address, which is the value of `c`. Therefore, printing `*p` is valid and works without any problem.

Conclusions

- A pointer can store only an address, not a normal integer value.
- Trying to assign an integer value to a pointer causes an error.
- The expression `*pointer` refers to an integer location, not an address.
- Trying to store an address in an integer location also causes an error.
- Assigning an address (like `&c`) to a pointer is correct because the types match.
- Once a pointer holds a valid address, using `*pointer` to store or access an integer value is valid.

TITLE : CALL BY VALUE AND CALL BY REFERENCE

OBJECTIVES

- To understand the difference between call by value and call by reference
- To learn how values are passed to functions in C
- To implement swapping of two numbers using both methods

THEORY

Call by Value

In call by value, a copy of the actual arguments is passed to the function.
Any changes made inside the function do not affect the original variables.

In this method:

- Separate memory is created for function parameters.
- Original values remain unchanged.

Call by Reference

In call by reference, the address of the actual arguments is passed to the function.
Any changes made inside the function affect the original variables.

In this method:

- The function works with the original memory location.
- Changes reflect outside the function.
- Pointers are used.

Problem I

Swap Two Numbers Using Call by Value

Source Code

```c
#include <stdio.h>

void swap(int x, int y)
{
    int temp;
    temp = x;
    x = y;
    y = temp;
}

int main()
{
    int a = 10, b = 20;

    printf("Before swapping: a = %d, b = %d\n", a, b);

    swap(a, b);

    printf("After swapping: a = %d, b = %d\n", a, b);

    return 0;
}
```

OUTPUT

Before swapping: a = 10, b = 20
After swapping: a = 10, b = 20

Problem II

Swap Two Numbers Using Call by Reference

Source Code

```c
#include <stdio.h>

void swap(int *x, int *y)
{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}

int main()
{
    int a = 10, b = 20;

    printf("Before swapping: a = %d, b = %d\n", a, b);

    swap(&a, &b);

    printf("After swapping: a = %d, b = %d\n", a, b);

    return 0;
}
```

OUTPUT

```
Before swapping: a = 10, b = 20
After swapping: a = 20, b = 10
```

CONCLUSION

- In call by value, changes made inside the function do not affect the original variables.
- In call by reference, changes made inside the function directly modify the original variables.
- Call by reference is useful when we want a function to return multiple results or modify actual data.

TITLE: INTRODUCTION TO STRUCTURE

OBJECTIVES

To understand the concept of structure in C

To learn how to define and use structures

To store and manipulate student details using structures

THEORY

Structure

A structure is a user-defined data type in C that allows grouping of different types of data under a single name.

Structures are used to represent a record. For example, a student record may contain:

- Name
- Roll number
- Marks

Syntax

```
struct structure_name
{
  data_type member1;
  data_type member2;
  ...
};
```

To access structure members, the dot (.) operator is used.

PROBLEM I

WAP TO STORE STUDENT DETAILS AND DISPLAY IT TO USER

```c
#include <stdio.h>

struct student
{
    char name[50];
    int roll;
    float marks;
};

int main()
{
    struct student s;

    printf("Enter Name: ");
    scanf("%s", s.name);

    printf("Enter Roll Number: ");
    scanf("%d", &s.roll);

    printf("Enter Marks: ");
    scanf("%f", &s.marks);

    printf("\nStudent Details:\n");
    printf("Name: %s\n", s.name);
    printf("Roll Number: %d\n", s.roll);
    printf("Marks: %.2f\n", s.marks);

    return 0;
}
```

PROBLEM II: WAP TO STORE STUDENT DETAILS AND DISPLAY THE DETAILS OF STUDENT WITH THE HIGHEST MARKS

```
#include <stdio.h>
struct student
{
    char name[50];
    int roll;
    float marks;
};

int main()
{
    int i;
    float max;
    struct student s[10], top;

    printf("Enter details of 10 students (Name Roll Marks):\n");

    for(i = 0; i < 10; i++)
    {
        scanf("%s %d %f", s[i].name, &s[i].roll, &s[i].marks);
    }

    max = s[0].marks;
    top = s[0];

    for(i = 1; i < 10; i++)
    {
        if(s[i].marks > max)
        {
            max = s[i].marks;
            top = s[i];
        }
    }

    printf("\nHighest Marks Student:\nName: %s Roll: %d Marks: %.2f\n",
           top.name, top.roll, top.marks);

    return 0;
}
```

PROBLEM III: WAP TO STORE STUDENT DETAILS AND DISPLAY THEM BACK IN ASCENDING ORDER OF MARKS

```
#include <stdio.h>
struct student
{
    char name[50];
    int roll;
    float marks;
};

int main()
{
    int i, j;
    struct student s[10], temp;

    printf("Enter details of 10 students (Name Roll Marks):\n");

    for(i = 0; i < 10; i++)
    {
        scanf("%s %d %f", s[i].name, &s[i].roll, &s[i].marks);
    }

    for(i = 0; i < 9; i++)
    {
        for(j = i + 1; j < 10; j++)
        {
            if(s[i].marks > s[j].marks)
            {
                temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }

    printf("\nStudents in Ascending Order of Marks:\n");

    for(i = 0; i < 10; i++)
    {
        printf("Name: %s Roll: %d Marks: %.2f\n",
               s[i].name, s[i].roll, s[i].marks);
    }

    return 0;
}
```

CONCLUSION

- Structures allow grouping of related data into a single unit.
- They help manage records like student details efficiently.
- Using structures, we can easily perform operations such as finding the highest marks and sorting records.

TITLE: FILE HANDLING IN C

OBJECTIVES

- To understand the concept and purpose of file handling in C.
- To learn how to open, read, write, append, and close files.
- To differentiate between text and binary files.
- To perform file operations using structures for storing and retrieving data.
- To understand the use of different file modes (`r`, `w`, `a`, `rb`, `wb`, `ab`).

THEORY

Definition:
File handling is the process of storing and retrieving data on secondary storage (like a hard disk) using files.
In C programming, files are used to permanently store data so that it can be accessed even after the program terminates.

Types of Files

1. Text Files
   - Contain human-readable characters.
   - Data is stored in lines and can be read or written using functions like `fprintf()`, `fscanf()`, `fgets()`, `fputs()`.

2. Binary Files
   - Contain data in binary format (machine-readable).
   - Can store any type of data, including structures.
   - Read/write using `fwrite()` and `fread()`.

File Modes in C

To work with files, you must open a file in a specific mode:

Mode	Description	Example
`"r"`	Read (file must exist)	`fopen("file.txt", "r")`
`"w"`	Write (creates new file or overwrites existing)	`fopen("file.txt", "w")`
`"a"`	Append (writes at end of file)	`fopen("file.txt", "a")`
`"rb"`	Read binary	`fopen("file.bin", "rb")`
`"wb"`	Write binary	`fopen("file.bin", "wb")`
`"ab"`	Append binary	`fopen("file.bin", "ab")`

Steps involved in File Handling:

- Open a file using fopen() in the desired mode.
- Read or write data using appropriate functions (fprintf, fscanf, fwrite, fread).
- Close the file using fclose() to save data and free resources.

SOME FILE HANDLING FUNCTIONS

Function	Description	Use Case
`fopen()`	Opens a file and returns a `FILE` pointer.	Starting any file operation.
`fclose()`	Closes an open file.	Saving changes and freeing memory.
`fputc()`	Writes a single character to a file.	Character-by-character logging.
`fgetc()`	Reads a single character from a file.	Parsing files symbol by symbol.
`fputs()`	Writes a string to a file.	Writing simple lines of text.
`fgets()`	Reads a line or string from a file.	Reading text file content safely.
`fprintf()`	Writes formatted data to a file.	Saving variables (like `%d`, `%f`).
`fscanf()`	Reads formatted data from a file.	Extracting specific data types.
`fseek()`	Moves the file pointer to a specific location.	Random access/skipping data.
`ftell()`	Returns the current position of the pointer.	Finding file size.
`rewind()`	Resets the file pointer to the start.	Re-reading a file from the top.
`feof()`	Checks if the End-of-File has been reached.	Controlling "while" loops for reading.

PROBLEM I : WAP Demonstrating text file handling usng fprintf and fscanf

```c
#include <stdio.h>

int main() {
    FILE *fp;
    char name[50];
    int roll;
    float marks;

    // Write to file
    fp = fopen("STUDENT.TXT", "w");
    for(int i = 0; i < 3; i++) {
        printf("Enter Name Roll Marks: ");
        scanf("%s %d %f", name, &roll, &marks);
        fprintf(fp, "%s %d %.2f\n", name, roll, marks);
    }
    fclose(fp);

    // Read from file using EOF
    fp = fopen("STUDENT.TXT", "r");
    printf("\nStudents from file:\n");
    while(fscanf(fp, "%s %d %f", name, &roll, &marks) != EOF)
        printf("Name: %s Roll: %d Marks: %.2f\n", name, roll, marks);
    fclose(fp);

    return 0;
}
```

PROBLEM II WAP DEMONSTRATING Binary File Handling with fwrite / fread

```
#include <stdio.h>

struct student {
    char name[50];
    int roll;
    float marks;
};

int main() {
    FILE *fp;
    struct student s;

    // WRITE MODE
    fp = fopen("STUDENT_STRUCT.BIN", "wb");
    if(fp == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    printf("Enter Name Roll Marks to store (Write Mode): ");
    scanf("%s %d %f", s.name, &s.roll, &s.marks);

    fwrite(&s, sizeof(s), 1, fp);  /    fclose(fp);

    // APPEND MODE
fp = fopen("STUDENT_STRUCT.BIN", "ab");


    printf("Enter Name Roll Marks to append (Append Mode): ");
    scanf("%s %d %f", s.name, &s.roll, &s.marks);

    fwrite(&s, sizeof(s), 1, fp);
 fclose(fp);

    // READ MODE
    fp = fopen("STUDENT_STRUCT.BIN", "rb");
    printf("\nStudents in file (Read Mode):\n");
    while(fread(&s, sizeof(s), 1, fp) == 1) \
        printf("Name: %s Roll: %d Marks: %.2f\n", s.name, s.roll, s.marks);

    fclose(fp);
    return 0;
}
```

OBSERVATION AND CONCLUSION

- File handling in C allows permanent storage of data beyond program execution.
- Binary files efficiently store and retrieve structured data like student records.
- Write mode creates a new file or overwrites, append mode adds data without losing existing records.
- Using fread() and fwrite() ensures accurate storage and retrieval of multiple fields in a structure.
- Proper file handling (opening, closing, and using correct modes) is essential to maintain **data integrity**.